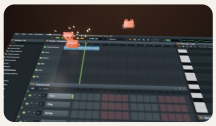
LOOPING CHARACTER FILM / 32 S / 16:9 / 30 FPS

Clawd cooks a beat.

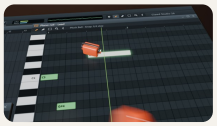
Clawd stands on an audio clip in an FL Studio-style project, spawns three subagents, and the four of them build a 150 BPM trap beat by landing on real controls: steps in the Channel rack, notes in the Piano roll, a fader and a filter in the Mixer. In the drop they play the song live, landing on every note as it sounds. Clawd exports it, the subagents merge back, File > New, and he drops the fresh clawd_beat.wav into the empty playlist: the beat of this loop is the sample at its start. The interface, the scene and the beat were all written by Claude Opus 5.5 from the prompt on page 2.



32.0 s	30 fps	1920x1080	146 hops	530	3
150 BPM, 20 BARS	960 FRAMES, LOOPED	CYCLES ON THE GPU	EACH ONE A NOTE OR A CLICK	WORDS OF PROMPT	FOLLOW-UPS



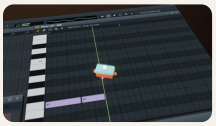
2.20 S
SPAWN



4.30 S
LEAD



8.30 S
DRUMS



10.60 S
808



16.03 S
DROP



27.63 S
EXPORT

THE FORMULA

01 Every landing is a sound
Hops land on the exact 16th where their step, note or click sounds: the score drives the dance.

02 The UI is live HTML
One screenshot per frame: steps light, notes grow, the playhead runs, menus and the dialog open.

03 Subagents, literally
Clawd spawns three helpers with one job each, and they merge back into him when the work is done.

04 The loop is the sample
The exported beat is the clip the next pass starts on, filtered: the end makes the beginning.

05 Meters read the mix
Mixer meters and the scope are computed from the WAV frame by frame, so they move with the beat.

06 Measure, don't guess
Contact sheets before the full render, loudness per section, a pixel diff across the seam.

PAGE 02 — **The prompt**, copy-ready.
PAGE 03 — **Every follow-up**, in order.
PAGE 04 — **Make it yours**, render it fast.
PAGE 05 — **Every hit** on its frame.
PAGE 06 — **The beat**, prompt and check.

The prompt

COPY-READY 530 WORDS

Paste it as is into Claude Code running Opus 5.5 at effort high, with Blender 5, Node with puppeteer, ffmpeg and Python with numpy, scipy and pedalboard installed. It writes the interface as HTML, the scene and the beat as Python, renders headless on the GPU and checks its own stills.

```
BRAND
HERO: Clawd, the Claude Code mascot as one smooth toy: body 12x8x6 units, two side nubs, four short legs, black eye slits, soft bevels, terracotta D97757. Three smaller subagent Clawds: the melodist in big headphones, the drummer in shades and a red sweatband, the bassist in a teal beanie with a pompom
APP: a faithful FL Studio-style dark workspace, no logos: menu and transport bar with tempo and song-position LCDs and a scope, Browser, Playlist with tracks and clips, a floating Channel rack with 16-step rows, a Piano roll, a Mixer with meters, faders and an FX chain, File > Export and the render dialog
RESULT: a 150 BPM trap/rage beat in F minor, built live: lead, drums, 808, build, drop
SET: every window is a floating slab over a dark warm studio; the UI glows like a screen

FORMAT: Blender 5 Python, Cycles GPU, 1920x1080, 30 fps, 32.0 s = 20 bars at 150 BPM, a seamless loop: frame 961 is frame 1. Build the interface as an HTML page and screenshot it with puppeteer once per frame into a 3840x2160 texture. One timeline.py holds the layout in UI pixels, the score in 16th steps and every hop; the HTML, the Blender scene and the audio all read it, so each landing hits on its step.

BEATS (seconds):
0-3.2 Clawd stands on clawd_beat.wav in the playlist while a filtered sample plays; on beats 1 to 3 of bar 2 he spawns three subagents, who leap to their windows
3.2-5.6 he mutes the sample; the melodist paints eight lead notes in the Piano roll, each sounding as it lands; Clawd adds Fruity Reverb 2
6.4-9.4 the drummer lights kick and clap steps, rides the hi-hat row with the playhead, then the open hat
9.6-12.4 the Piano roll switches to the 808; the bassist paints it, glides included
12.8-16 build: Clawd rides the Fruity Filter cutoff, the drummer hops a clap roll, the melodist wakes the Saw channel; all four leap into the drop
16-25.6 the drop: each subagent plays its part live by landing on every note and step as it sounds; Clawd bounces on the master fader
25.8-32 File > Export > Wave file, Start; the subagents dive back into Clawd as the bar fills; tape stop, File > New, he drags the fresh clawd_beat.wav into the empty playlist and stands on it, where the film began

LOOP: the exported beat is the sample the film opens on, filtered; the fresh project equals frame 1, and the playhead, the music and the camera all wrap.

CAMERA: follow the active Clawd at about 50 degrees, f/8, smoothed so hops do not shake the frame; wide only for the spawn and the drop; keep the whole export in one shot.

LOOK: UI as emission under dark glossy glass; keep the lights out of the UI reflections; orange and teal rims that pump with the kick in the drop; bloom, 0.5 shutter blur, AgX; the body must read D97757.

CHECKS: render the UI frames first and diff frame 960 against frame 1; then two dozen stills at half res as contact sheets, and fix the camera before the full render.
```

BRAND	**The block you swap.** HERO, APP, RESULT, SET. The rest of the prompt only uses these names.
FORMAT	Bars, not seconds: 20 bars at 150 BPM is exactly 960 frames, so the music and the picture loop together.
BEATS	Written as a production session: melody, drums, 808, build, drop, export. Every hop is the next real step.
LOOP	The line that makes it endless: the export becomes the sample, so the end state is the first frame.
CAMERA	Following a character that hops every 0.3 s shakes the frame; the smoothing line is what keeps it calm.

Every follow-up

3 ROUNDS

After each contact sheet I wrote what I saw and what to change, in order. These are the notes; the pictures below are the same frame before and after follow-up 1.

FOLLOW-UP 1 / AFTER THE FIRST 23 STILLS

```
The camera is too low: the interface reads as a grazing slab, the wide shots turn
the Clawds into specks and depth of field blurs half the frame. Raise every shot
to about 50 degrees, come closer, f/8. The body reads peach: deepen the orange
and drop the key light to 500 W.
```

FOLLOW-UP 2 / AFTER THE SPAWN STILLS

```
The subagents leave the top of the frame on their first leap and the camera
reaches the Piano roll late. Lower the first-leap apexes to 0.3 m, arrive by 3.05
s, and follow the characters through a 1.4 s smoothing so the camera does not
bounce with every hop.
```

FOLLOW-UP 3 / AFTER THE EXPORT STILLS

```
Clawd's leap from the render dialog back to File crosses 865 px and the camera
smears. Move the dialog next to the File menu so the export, the three merges and
File > New stay in one shot; re-render UI frames 760 to 905 only.
```

FOLLOW-UP 1

Result: the Clawds fill the frame, the UI reads as an app, the body reads terracotta (pictures below).

FOLLOW-UP 2

Result: every spawn lands in shot and the melodist is framed on his first note.

FOLLOW-UP 3

Result: the whole export plays in one steady shot, merges included.

FULL RENDER

Result: 960 frames at 1920x1080, 128 samples with OptiX denoise, about 1.6 s per frame on an RTX 4070 SUPER: 25 minutes for the film, plus 960 UI screenshots at 3840x2160.

FOLLOW-UP 1 IN ONE PICTURE: 16.03 S, THE DROP



Before: a grazing wide shot, the Clawds are specks.

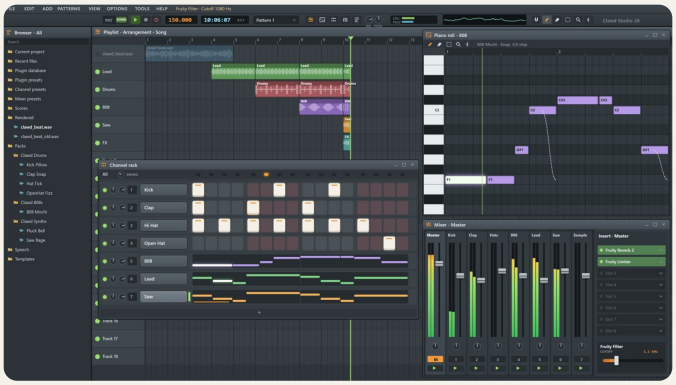


After: 50 degrees and closer, the drop reads at a glance.

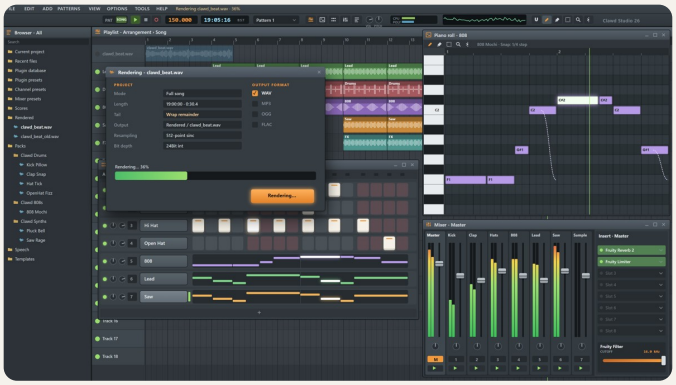
Make it yours

Only the BRAND block changes. Below: two frames of the interface exactly as Opus built it before any 3D. Swap APP for any program and RESULT for what the crew should make in it.

```
BRAND (an example swap, not rendered)
HERO: the same
APP: a Logic Pro-style workspace: Tracks area with regions, Library, Step Sequencer, Piano Roll, a Mixer with channel strips,
Smart Controls, File > Bounce > Project and the Bounce dialog
RESULT: a 122 BPM lo-fi house groove: chords, four-on-the-floor, a walking bass, a filtered drop
SET: the same
```



Frame 401, 3840x2160: the build, the cutoff ride at 1.1 kHz.



Frame 831: the render dialog at 33%, Tail: Wrap remainder.

Render it fast

1. Write the beat first: `python audio.py` also writes the meter levels and the scope trace for the UI.
2. Build the UI: `python build_ui.py`, then `node render_ui.js` for the 960 UI frames.
3. Build the .blend from scene.py with `blender -b --factory-startup -P scene.py`.
4. Preview two dozen stills at 50% and 40 samples, plus frames 959, 960 and 1, and read them as contact sheets.
5. Pass `-P gpu.py` on every render: the OptiX device is a user preference, not part of the file.
6. Encode H.264 High CRF 17, 30 fps, soft vignette, mux the WAV as AAC 320k, +faststart.

Every hit on its frame

Read straight from timeline.py; the frame is time x 30 + 1. At 150 BPM a beat is 0.4 s and a 16th is 0.1 s, so every landing sits on a step of the grid.

TIME	FRAME	PICTURE	SOUND
0.00 to 1.60 s	f1 to f48	Clawd on clawd_beat.wav, the playhead runs	the filtered sample, vinyl crackle
1.60 / 2.00 / 2.40 s	f49 / f61 / f73	three subagents pop out of Clawd	rising pops and chimes, a small riser
3.20 s	f97	Clawd mutes the sample	click, the sample stops
3.20 to 5.60 s	f97 to f169	the melodist paints 8 lead notes	each pluck as its note lands
4.40 s	f133	Fruity Reverb 2 on slot 1	chime, the lead opens up
6.40 to 9.40 s	f193 to f283	kick, clap, hi-hat ride, open hat	each step sounds as it lights
9.60 to 12.40 s	f289 to f373	the bassist paints the 808, two glides	each 808 note as it lands
12.80 to 15.20 s	f385 to f457	Clawd rides the cutoff, the clap roll	filter opens 500 Hz to 16 kHz, riser
16.00 s	f481	all four land: the drop	boom, crash, supersaws, 808
16.00 to 25.60 s	f481 to f769	they play every note and step live	the full beat, hat rolls every 2 bars
25.80 to 27.20 s	f775 to f817	File > Export > Wave file, Start	menu swishes, clicks
27.60 / 28.00 / 28.40 s	f829 / f841 / f853	subagents merge back into Clawd	falling pops and chimes
28.80 s	f865	export done, tape stop	the beat slows to a halt
29.30 s	f880	File > New clears the project	a downward sweep
29.75 to 30.40 s	f893 to f913	he drags clawd_beat.wav into the playlist	whoosh, reverse cymbal, the sample

3 TIPS

1

Steps are the API

Write the score in 16th steps; one function turns steps into seconds, pixels and frames.

2

Land on the note

A hop that lands 0.1 s early reads as wrong: the take-off is computed back from the landing.

3

Show the software logic

Window titles, channel names, FX slots, the render dialog: producers recognise every move.

The beat

COPY-READY 200 WORDS

The picture comes from the prompt on page 2; the beat came from this one more prompt, driven by the same timeline.py.

```
Add the beat. Synthesize everything in Python with numpy, scipy and pedalboard: no samples, no words. 48 kHz stereo WAV, 32.0 s, a seamless loop: render three loops and keep the middle one. Read every event from timeline.py; a note or step sounds from the moment a subagent lands on it.

Music: 150 BPM trap/rage in F minor on Fm and Db. A bell pluck lead in a 3-3-2 rhythm; a punchy kick on steps 1, 7 and 11, a layered clap on 9, 808-style hi-hats with 32nd and triplet rolls, an open hat; a distorted 808 with a knock and two glides; a build with a clap roll, a noise riser and a lowpass that opens as Clawd rides the cutoff; a drop with detuned 7-voice supersaw stabs and the lead on supersaws, sidechained to the kick; a tape stop when the export ends. The opening sample is the drop itself, lowpassed with crackle.

Effects: pops and chimes for spawns and merges, clicks for every button, menu swishes, render ticks, a sweep for File > New, a whoosh for the dragged file.

Master: headroom into a gentle compressor and a limiter at -1.2 dBFS. Also write per-frame meter levels for the mixer.
```

The checks I ran

LOUDNESS PER SECTION AND THE SEAM: THREE SOUND FOLLOW-UPS

1. Peaks of +10 dB in the 808 and kick stems: the block-wise sweeping filter reset its state. Replaced it with a crossfaded bank of fixed lowpasses. 2. The limiter was eating 7 dB of the drop: 0.62 headroom in front of it, now the intro sits at -18 and the drop at -12 dBFS RMS. 3. Hi-hat edges jumped 1.4 between samples: a 14 kHz lowpass, a 1.8 ms attack and -3 dB. The seam jump is 0.005, below the median sample step.

Into the MP4: the WAV was muxed in as the video's AAC 320k track with ffmpeg; the same step works for your own video.